Three LLM agents, one refactoring kit: four code smells in Apache Commons CLI 1.9.0

SWE 539 Homework 2 · Ali Aljowaher · November 2026

1 Setup

We asked three LLM coding agents to remove the same four code smells from Apache Commons CLI 1.9.0 (tag `rel/commons-cli-1.9.0`, commit `698b238`): two within-class smells (W01, W04) and two between-class smells (B01, B02), each with the refactoring technique fixed in advance by the reviewed smell sheet (Table 2). A starter kit (`kit.py`) did every mechanical step in the same way for every run. It cloned the tag onto a fresh branch, saved the original code and ran the project's own test suite on the untouched code (797 JUnit tests: 738 pass, 59 skipped, 0 fail). After the agent's edits it re-ran the same, unmodified suite, compared the public and protected API, committed the result, logged everything to one Excel workbook and measured both versions with CK. The agent only had to plan, refactor in small tested steps and document its process. The protocol gave every run the same unified prompt, with only the smell ID, LLM name and model filled in, plus the same task file, the same rules and the same repair budget of at most three `finish` attempts (Table 1). The deviations from this protocol are listed in Section 4.

Table 1: Experimental setup.

Subject	Apache Commons CLI 1.9.0, `src/main/java`; one fresh branch per smell and LLM, all starting from the release tag
Oracle	Original JUnit suite, 797 tests (738 pass, 59 skipped); the baseline is run once per commit and reused
Verdict	PASS = compiles, 0 regressions, no test file changed, no public/protected API removed, code actually changed
Metrics	CK 0.7.0 on the whole tree, summarised over the classes of the affected files: Size = LOC (sum), Cohesion = LCOM* (mean; 0 = cohesive, 1 = no cohesion), Coupling = CBO (sum)
Toolchain	OpenJDK 17.0.21, Maven 3.9.6, Git 2.39.5, Python 3.11 (`kit.py`, openpyxl)
LLM agents	**Claude Opus 5.5** (`claude-opus-5-5`, Claude Code CLI) · **Gemini 3.1 Pro** (agent tool not recorded) · **Copilot**, recorded as "Copilot SDK in VS Code (2026-11-10)" (underlying model not recorded); all in agent mode, running the kit's commands themselves

Table 2: The four smells and the refactoring every LLM had to apply.

ID	Type	Smell	Location	Prescribed refactoring
W01	Within	Long Method	`HelpFormatter.appendOptions`	Extract Method (×3); Replace Temp with Query where practical
W04	Within	Long Parameter List	`HelpFormatter.printHelp` (9 parameters)	Introduce Parameter Object; Preserve Whole Object
B01	Between	Feature Envy	`HelpFormatter`, `OptionGroup` → `Option`	Extract Method, then Move Method (to Option)
B02	Between	Shotgun Surgery	`Options`, `DefaultParser`, `Parser`, `MissingOptionException`	Extract Class + Move Method + Move Field

2 Results

All 12 runs ended with verdict PASS (12 of 12). Each compiled, kept the 738 originally passing tests green with 0 regressions in total, left the test files untouched and removed no public or protected API element (Table 3). Every run passed at its first `finish`, and 11 of the 12 used only that one attempt. Claude's W04 ran `finish` a second time, only to complete its `process.md`; no code changed. Correctness therefore does not separate the three LLMs. The differences lie in how far each agent carried the prescribed technique (Fig. 1) and in what the metrics then report (Fig. 2).

3 Differences between the LLMs

W01 Long Method. The three refactorings are practically identical. The task names the three helpers to extract (`buildOptionLabel`, `appendSinceHeader`, `appendOptionRow`), and every agent used exactly those names. `appendOptions` shrank from 66 to 29 LOC (complexity 20 → 7) in all three runs, and the class-level CK numbers coincide. The runs differ only in the helpers' parameter lists (Gemini and Copilot pass a `StringBuilder`, Claude a `String`) and by three diff lines.

W04 Long Parameter List. All three introduced a three-field layout object (width, left pad, description pad) and turned the public 9-parameter `printHelp` into a one-line delegate (complexity 7 → 1), keeping every public overload. They differ in scope. Claude routed only `printHelp` through a new top-level, package-private class `HelpLayout` (1 new member). Gemini built the same class and also added a private `printOptions` overload that takes it. Copilot used a nested `HelpFormatter.Layout` and added `Layout`-taking overloads of `printOptions`, `renderOptions` and `appendOptions`, moving the 66-line body of `appendOptions` into the new overload (9 new

Table 3: All twelve runs; the verdict is PASS in every row. Tests = the original suite on the refactored code (the same 59 tests are skipped on the original code). Att. = `finish` attempts. Diff lines include the license header and Javadoc of new files. Members = methods, fields and declarations changed / new / removed. Metrics are CK values before → after.

Smell	LLM	Tests pass/total	Regr.	Att.	Diff lines +/−	Members chg/new/rem	New type	Size LOC, sum	Cohesion LCOM*, mean	Coupling CBO, sum
W01	Claude	738/797	0	1	+90/−41	1/3/0	–	425→435 (+10)	0.955→0.957	9→9
	Gemini	738/797	0	1	+90/−41	1/3/0	–	425→435 (+10)	0.955→0.957	9→9
	Copilot	738/797	0	1	+87/−41	1/3/0	–	425→435 (+10)	0.955→0.957	9→9
W04	Claude	738/797	0	2	+96/−5	1/1/0	HelpLayout	425→447 (+22)	0.955→0.728	9→10
	Gemini	738/797	0	1	+116/−15	1/2/0	HelpLayout	425→451 (+26)	0.955→0.728	9→10
	Copilot	738/797	0	1	+43/−14	4/9/0	Layout (nested)	425→457 (+32)	0.955→0.479	9→10
B01	Claude	738/797	0	1	+82/−37	4/4/0	–	845→855 (+10)	0.874→0.876	18→18
	Gemini	738/797	0	1	+78/−41	4/4/0	–	845→845 (0)	0.874→0.874	18→18
	Copilot	738/797	0	1	+38/−23	4/2/0	–	845→844 (−1)	0.874→0.874	18→18
B02	Claude	738/797	0	1	+101/−61	10/1/2	RequiredOptions	676→675 (−1)	0.729→0.562	32→34
	Gemini	738/797	0	1	+91/−42	9/1/0	RequiredOptions	676→681 (+5)	0.729→0.575	32→34
	Copilot	738/797	0	1	+97/−48	10/2/0	RequiredOptions	676→693 (+17)	0.729→0.689	32→34

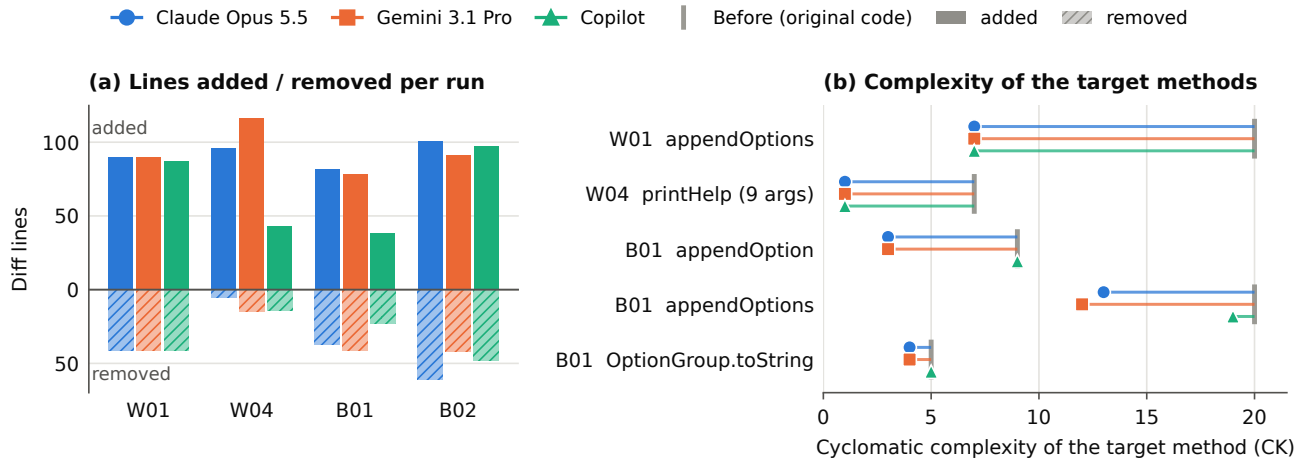


Figure 1: (a) Diff lines per run. New files count in full, so most of Claude's W04 +96 is the new `HelpLayout` file, whereas Copilot's +43 adds 9 members. (b) Cyclomatic complexity (CK) of the smell's target methods: grey tick = original, markers = after each LLM's refactoring. Rows where a method's body moved into another member are left out (Copilot's W04 `appendOptions`, B02's `updateRequiredOptions`).

members). In every run `HelpFormatter` itself became marginally less cohesive (LCOM* 0.955 → 0.956 / 0.957 / 0.958 for Claude / Gemini / Copilot) and gained one dependency (CBO 9 → 10).

B01 Feature Envy. All three moved description and label logic from `HelpFormatter` into `Option` (`getDescriptionOrEmpty` plus label builders) and shortened `OptionGroup.toString` (25 → 18 LOC). Claude and Gemini moved 4 methods each and cut `HelpFormatter` from 425 to 397 and 396 LOC. `appendOption`'s complexity fell from 9 to 3, and `appendOptions`' from 20 to 13 (Claude) or 12 (Gemini). Copilot moved only 2: `HelpFormatter` shrank only to 415 LOC, `appendOption` kept complexity 9 and `appendOptions` went to 19. `Option` absorbed the moved code (+45 / +36 / +16 LOC); coupling did not change in any run.

B02 Shotgun Surgery. All three extracted a package-private `RequiredOptions` class (the name the task suggests) that owns the per-parse list of still-missing required options, and kept the public and protected API by delegating to it. Claude went furthest. It deleted both private `updateRequiredOptions` clones (2 removed members) and moved their logic into `RequiredOptions.markSeen`, whereas Gemini and Copilot kept them as thin delegates. Copilot also changed the required-option bookkeeping in `Options` (CBO 4 → 5) and wrote a larger tracker (38 LOC against 25). `Parser`'s cohesion improved most in Claude's run (LCOM* 0.692 → 0.583), less in Gemini's (0.641), and got worse in Copilot's (0.712). `DefaultParser` got slightly worse in all three (0.797 → 0.803 / 0.809 / 0.809).

What the metrics say. The large cohesion gains are mostly an artefact of averaging (Fig. 2). Because the reported value is a mean over the counted classes, adding one small, cohesive class pulls it down even when no existing class improves. Copilot's W04 mean falls from 0.955 to 0.479, yet `HelpFormatter`'s own LCOM* rose slightly in every W04 run. The only clear cohesion improvement of an existing class is `Parser` in Claude's and Gemini's B02. Coupling rises by +1 in W04 because `HelpFormatter` now depends on the layout class (the layout classes themselves have CBO 0). It rises by +2 in B02 because the new tracker is counted (CBO 4, Copilot 3), while each existing parser lost one dependency (`Parser` CBO 12 → 11). Size changes are small (between −1 and +32 LOC). Per-class and per-method values (`metrics.md` in each run folder) are therefore the right basis for comparing the LLMs, not the three headline numbers.

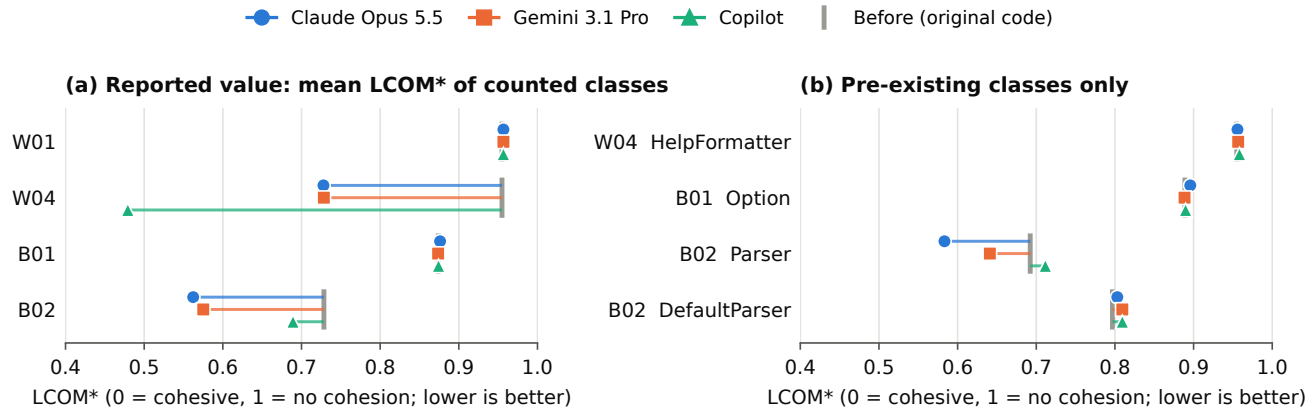


Figure 2: (a) The Cohesion value the kit reports: mean LCOM* over the counted classes, including the classes a refactoring creates. (b) The same metric for the main pre-existing classes. The new classes are `HelpLayout` (LCOM* 0.500; Claude, Gemini), `HelpFormatter.Layout` (0.000; Copilot) and `RequiredOptions` (0.000 Claude, 0.000 Gemini, 0.500 Copilot).

4 Threats to validity

- **Model identity.** Copilot's underlying model and Gemini's agent tool were not recorded. Copilot's four runs were logged under three LLM names (`Copilot`, `Copilot-W01`, `Copilot-B01`) and are merged here. Its W04 and B02 ran concurrently in two separate clones.
- **Session hygiene.** Claude's W01 and B01 ran back to back in one session that was told to read the README and prompt files, rather than in fresh sessions with the filled-in prompt pasted. The metrics step was added to the kit after those two runs, and their CK values were measured afterwards with the same command. Claude's W04 and B02 ran as fresh sub-agents launched from an operator's Claude Code session, with the unmodified prompt. All runs shared one kit folder, so earlier runs' outputs were readable by later agents. Gemini's W04 and B02 ran after Claude's and produced classes with identical CK profiles (`HelpLayout`: 19 LOC, WMC 4; `RequiredOptions`: 25 LOC, WMC 8, CBO 4). Convergence under a prescriptive task is plausible, but reuse cannot be ruled out.
- **Prescriptive task.** The smell sheet names the helpers and classes for W01, B01 and B02, which limits how different the solutions can be.
- **Oracle and sample.** PASS means the existing suite (59 tests skipped) saw no change in behaviour, not that the smell is gone. Each smell was run once per LLM, so LLM nondeterminism is not measured. Wall-clock times include human approval waits and are not compared.

5 Conclusion

All three agents delivered behaviour-preserving refactorings for all four smells at the first attempt, so correctness did not distinguish them. They differed in scope. Claude and Gemini made nearly the same choices, with Claude the more minimal in W04 and the more thorough in B02. Copilot was the most conservative in B01 (complexity of `appendOption` unchanged) and the most expansive in W04 (9 new members). Existing classes changed little in cohesion and coupling. The big drops in the headline Cohesion value come from averaging in new classes, so any judgement of "better code" should rest on the per-class numbers and on reading the diffs.